

GUIDE — Guide d'utilisation complet de PBLCA

Ce guide décrit le pipeline complet, étape par étape : configuration du cas d'étude, exécution Python, analyse R, documentation générée et tests. Chaque section explique **ce que fait l'utilisateur, ce que fait le code**, et renvoie vers la documentation auto-générée (pdoc) et les fichiers sources.

Sommaire

1. Prérequis et installation
2. Architecture en trois couches (comprendre avant d'utiliser)
3. Étape 1 — Configurer le cas d'étude
4. Étape 1 bis — La card TOML et le CLI (zéro code)
5. Étape 2 — Exécuter le pipeline Python
6. Étape 3 — Analyses R
7. Étape 4 — Tests et contrat Python → R
8. Étape 5 — Documentation auto-générée
9. Pour aller plus loin (niveau développeur)

1 — Prérequis et installation

Python (≥ 3.10) :

```
cd pblca
pip install -e ".[dev]"      # moteur + pytest + pdoc
```

R (≥ 4.0) pour les analyses et figures :

```
install.packages(c("jsonlite", "ggplot2"))
```

Vérification rapide :

```
pytest -q                      # suite de tests (cf. Étape 4)
pdoc pblca -o docs_html        # documentation HTML (cf. Étape 5)
Rscript -e "library(jsonlite)" # analyse R
```

2 — Architecture en trois couches (comprendre avant d'utiliser)

Le moteur est conforme ISO 14040/14044 : l'inventaire (kg de gaz émis) est

séparé de la caractérisation (conversion en kg CO₂e). Trois couches :

COUCHE 3 — IMPACTS	pblca/impacts.py	GWP100, GWP20, GWP* (Cain 2019)
COUCHE 2 — FLUX DE GAZ	pblca/gases.py	registre CH ₄ /CO ₂ /N ₂ O, traçabilité
COUCHE 1 — PROCESSUS	pblca/processes/	entérique, fumier, sol N ₂ O, carbone, achats, travaux mécaniques

Deux transverses :

- pblca/params.py — [ParameterSet](#) :

chaque paramètre porte sa valeur centrale, son incertitude (log-normale),

son unité et sa **référence bibliographique** (exigence ISO : aucune valeur non sourcée). Les facteurs GWP (table 7.15 AR6) sont des paramètres comme les autres : leur incertitude se propage dans le Monte-Carlo.

- `pblca/registry.py` — registre de modèles : pour chaque `*slot*` (ex. `enteric_ch4`), plusieurs variantes interchangeables (Tier-2 2006, Tier-2 2019, Tier-3 Mills, Tier-3 Sauvart 2011, mesures AHCS...), chacune avec sa référence. Interface universelle : un slot se change sans toucher au reste.

L'`LCAEngine` (`pblca/engine.py`)

orchestre : sélection des variantes → exécution des processus → remplissage du `GasLedger` → calcul des impacts (`characterize`) → stockage JSON.

La sous-couche `pblca/mc.py` génère les tirages Monte-Carlo

(`ParameterSet.draw`) et perturbe

les rations mesurées ; les processus restent des évaluateurs purs.

Étape 1 — Configurer le cas d'étude

Fichier : `case_studies/ferme_20ha.py` (déclaratif, rien à coder).

Le cas d'étude est décrit par une `CaseStudyConfig` (définie dans `pblca/scenarios.py`, `CaseStudyConfig`) :

Champ	Rôle
<code>name</code>	identifiant du cas (préfixe des <code>sim_id</code> : <code>mc_<name>_...</code>)
<code>farm_builder</code>	fonction qui construit la ferme (groupes d'animaux, parcelles, systèmes fumier) ; voir <code>pblca/case_study.py</code> et <code>pblca/farm_spec.py</code>
<code>measurements</code>	mesures disponibles sur cette ferme : ingestions mesurées (<code>dmi_measured</code>), énergies (<code>ge_measured</code>), CH ₄ mesuré (<code>ch4_ahcs</code>), avec leurs erreurs relatives (<code>ration_rel_sd</code> , <code>ch4_ahcs_rel_sd</code>)
<code>variant_grid</code>	dictionnaire <code>*slot</code> → liste de variantes* à balayer (ex. 11 variantes entériques × 2 fumier)
<code>named_combinations</code>	chaînes cohérentes de variantes (ex. <code>inra_tier3</code> : Sauvart 2011 + Eugène 2019)
<code>mc</code>	options numériques : <code>n_iterations</code> , <code>seed</code> (reproductibilité)
<code>main_enteric_variant</code>	la méthode principale de référence pour l'analyse appariée (ex. <code>tier2_fao_ym_modelled_ingestion</code>) ; toutes les autres deviennent des alternatives comparées à elle
<code>paired_grid</code>	grille appariée multi-slots (<code>PairedGridConfig</code>) : variantes à balayer par slot, sélection de référence des différences appariées, <code>full_factorial</code> (produit cartésien) — voir Étape 2, point 5

Comment choisir `main_enteric_variant` : c'est la variante dont le

GWP100 est décomposé (Étape 3) et le point zéro des figures d'effet-modèle.

Choisissez la méthode que vous considérez comme votre estimation de référence au niveau ferme.

Étape 1 bis — La card TOML et le CLI (zéro code)

Fichier : `cards/ferme_20ha.toml` (déclaratif) — alternative à la

`CaseStudyConfig` Python pour les utilisateurs qui ne veulent pas écrire de code. Une étude complète (ferme, variantes, plan Monte-Carlo, sortie) tient dans **un seul fichier TOML**, validé au chargement (slot/variante inconnus → erreur explicite immédiate) :

```
pblca run cards/ferme_20ha.toml      # exécute l'étude décrite par la card
pblca slots [--slot enteric_ch4]     # inventaire des slots et variantes
pblca farms                          # fermes intégrées disponibles
```

Sections de la card (toutes optionnelles sauf la ferme) :

Section	Rôle
[study]	name ; farm = ferme intégrée (ferme_20ha) ou table [farm] complète en ligne ; named_combinations
[model_selection]	variantes du run central (slots absents = défaut du registre)
[variant_grid]	sweeps mono-slot : chaque variante est exécutée à valeurs centrales puis en Monte-Carlo
[paired_grid]	grille appariée multi-slots : slots (variantes par slot), main_selection (référence des différences appariées), full_factorial (produit cartésien au lieu d'un facteur à la fois)
[monte_carlo]	n_iterations, seed
[datastore]	path du JSON de résultats (défaut results.json)

Le fichier de résultats embarque la card elle-même (hachage SHA-256 + texte brut sous `card / card_source`) : chaque étude reste traçable et reproductible (ISO 14044 §4.5). Les variantes exigeant des mesures que la ferme ne possède pas sont exclues automatiquement, avec la raison. Le CLI produit la même sortie `results.json` que `run_case_study.py` — les analyses R de l'Étape 3 s'appliquent donc telles quelles.

Étape 2 — Exécuter le pipeline Python

```
python run_case_study.py
```

Le script orchestre (voir `pblca/scenarios.py`, fonction `run_case_study`, fonction `run_case_study`) :

1. **Grille de scénarios** — une exécution à valeur centrale par combinaison de variantes ; les combinaisons non exécutables (champs manquants) sont exclues avec la raison.
2. **Monte-Carlo par variante** — un MC indépendant par variante de la grille (`mc_<cas>_<slot>_<variante>`) : statistiques (mean, sd, p5, p50,

p95) des émissions entériques par lot, par tête/jour, du CH₄ fumier par système, et des impacts ferme.

3. **Comparaison appariée des rations** — à chaque itération, un seul tirage de paramètres, deux évaluations (équations IPCC vs rations mesurées) : la différence appariée isole l'effet du mode de ration.

4. **Grille appariée des variantes entériques** — le cœur de l'analyse de sensibilité :

```
LCAEngine.run_paired_variant_grid.
```

À chaque itération, **un seul tirage** de paramètres, puis toutes les variantes évaluées avec ce même tirage. Les colonnes du tableau sont donc directement comparables ligne à ligne : une différence entre deux colonnes d'une même ligne reflète le choix du modèle, pas le bruit d'échantillonnage.

5. **Grille appariée multi-slots** (si `paired_grid` est déclaré) — généralisation du point 4 à n'importe quel ensemble de slots :

```
LCAEngine.run_paired_grid
```

(`pblca/mc.py`, `run_paired_grid`). Un tirage par itération, puis

toutes les colonnes slot × variante évaluées avec ce même tirage — en mode **un facteur à la fois** (défaut : une colonne par variante balayée, les autres slots balayés restant à leur variante de référence) ou en **factoriel complet** (`full_factorial = true` : produit cartésien, ex. 11 entériques × 2 fumiers = 22 colonnes appariées). Les **différences appariées** entre deux colonnes ne différant que par un slot isolent l'effet pur de ce slot (décomposition de variance étendue au choix fumier, voir Étape 3.3).

`run_paired_variant_grid` est désormais un cas particulier de `run_paired_grid` (`slots = {"enteric_ch4": ...}`) ; son contrat de sortie (clés `variants`, `main_variant`, colonnes `<variante>__<indicateur>`) est inchangé — les scripts R existants ne bougent pas. L'entrée JSON correspondante est `sim_id = mc_<cas>_paired_grid`, avec les colonnes `<slot>=<variante>__<indicateur>` et la référence des différences dans `main_selection`.

Sortie : `results.json` — une entrée par simulation. L'entrée de la grille appariée (`sim_id = mc_<cas>_enteric_paired`) contient, sous `uncertainty` :

- `emissions_table` — une ligne par itération : `iteration`, une colonne

<variante>__<indicateur> par variante (indicateurs = gwp100, gwp20, gwpstar, ch4_kg, co2_kg, n2o_kg — totaux ferme), et les colonnes par **source × gaz de la variante principale** (enteric__ch4_kg, soil_n2o__n2o_kg, etc. — liste dans source_gas_columns) ;

- parameter_draws_table — la valeur tirée de **chaque** paramètre à chaque itération (traçabilité complète) ;
- central_gwp_factors — les facteurs GWP centraux (table 7.15 AR6), utilisés par l'analyse R pour séparer erreur d'inventaire et erreur de caractérisation ;
- farm_indicators_stats, paired_differences_stats — statistiques par variante et différences appariées contre la variante principale.

Cycle de vie de results.json : chaque exécution démarre sur un fichier vierge ; un results.json précédent est déplacé vers results_archive/results_<horodatage>.json (aucune perte, ISO 14044 §4.5 ; les figures R ne mélangent donc jamais deux exécutions). Le répertoire d'archive est ignoré par git, comme results.json.

Étape 3 — Analyses R

Toutes les figures et CSV vont dans R/ par défaut (ou le répertoire passé en second argument). Les scripts lisent results.json — régénéré après chaque modification de la configuration.

3.1 — Analyse de sensibilité appariée

```
Rscript R/analyse_enteric_sensitivity.R results.json
```

Quatre sorties complémentaires :

1. Corrélations paramètres → GWP100 —

enteric_paired_correlation.csv (Pearson + Spearman, par variante) et heatmap_fig_correlation_parametres_ch4.png. Chaque paramètre tiré est corrélé au GWP100 de chaque variante : la heatmap dit *quel paramètre pèse* sur chaque modèle. Deux panneaux : erreurs d'**inventaire** (kg de gaz émis) vs erreurs de **caractérisation** (facteurs GWP).

2. Effet pur du choix de modèle —

enteric_paired_model_effect.csv et fig_effet_modeles_gwp100.png : différence appariée de chaque alternative contre la variante principale, par indicateur. Deux barres d'erreur par alternative : erreur **totale** (facteurs GWP tirés) et erreur **inventaire seulement** (facteurs GWP centraux) ; l'écart entre les deux = contribution de la caractérisation.

3. Décomposition de variance du GWP100 —

enteric_variance_decomposition.csv et

fig_decomposition_variance_gwp100.png. Décomposition **exacte** par

covariance : chaque terme (source × gaz, scindé en partie inventaire et

partie caractérisation pour CH₄/N₂O) reçoit

$\text{Cov}(\text{terme}, \text{GWP100_total}) / \text{Var}(\text{GWP100_total})$; les contributions

somment à 100 % (aucune approximation, aucune hypothèse

d'indépendance). C'est la réponse directe à « *quelle erreur joue sur

le résultat final ?* » : elle montre, par exemple, que l'incertitude du

GWP100 ferme peut venir majoritairement du facteur CH₄ AR6 plutôt que

des paramètres d'élevage.

4. Tables brutes — enteric_paired_emissions.csv et

enteric_paired_parameter_draws.csv (exports des deux tables JSON),

pour toute analyse ad hoc.

Le résumé console affiche les 5 paramètres les plus influents par type

d'erreur, les effets de modèle et la décomposition de variance par bloc.

3.2 — Figures descriptives par variante

Script	Figure	Contenu
plot_enteric_ch4_groups.R	fig_ch4_par_lot.png	CH ₄ entérique par lot d'animaux (mean ± sd, p5–p95), par variante
plot_enteric_ch4_by_model.R	fig_ch4_par_modele.png	CH ₄ entérique ferme par variante
plot_enteric_ch4_per_head_day.R	fig_ch4_par_tete_jour.png	émissions par tête et par jour (comparaison inter-modèles)
plot_manure_ch4_by_model.R	fig_ch4_fumier_par_modele.png	CH ₄ fumier par système de gestion, par variante
plot_ration_comparison_correlation.R	fig_correlation_rations.png	corrélation équations IPCC vs rations mesurées

3.3 — Analyse de la grille appariée multi-slots

Rscript R/analyse_paired_grid.R results.json [output_dir]

Lit l'entrée `sim_id = mc_cas_paired_grid` (étape 5 du pipeline, un

tirage par itération partagé par toutes les colonnes de la grille) :

1. Corrélations paramètres → GWP100 par colonne —

paired_grid_correlation.csv : Pearson + Spearman, avec le type

d'erreur (inventory / characterisation).

2. Effet pur du choix de modèle, par slot —

paired_grid_model_effect.csv et

fig_effet_slots_gwp100.png (forest plot facetté par slot) :

différence appariée de chaque variante balayée contre la sélection

de référence (`main_selection`), par indicateur. Exemple de lecture : « passer du Tier-2 au Tier-3 fumier change le GWP100 ferme de $X \pm Y$ kg CO₂e, indépendamment des paramètres ».

3. Décomposition de variance appariée à deux facteurs —

`paired_grid_variance_share.csv` (+ `fig_interaction_slots_gwp100.png`) : part du GWP100 imputable à chaque choix de modèle (par slot balayé), à leur **interaction** (l'effet d'un choix dépend-il de l'autre ? identifiée uniquement en factoriel complet) et aux tirages appariés (résidu). En mode *un facteur à la fois*, la décomposition est additive et un avertissement indique de déclarer `full_factorial = true` pour identifier l'interaction.

4. Tables brutes — `paired_grid_emissions.csv` et

`paired_grid_parameter_draws.csv`.

Étape 4 — Tests et contrat Python → R

```
python -m pytest tests/          # 136 tests
```

- **Tests unitaires** (`tests/test_engine.py`, `tests/test_mc.py`, ...) :

cohérence des équations (re-calcul indépendant des formules IPCC/Tier-3), idempotence des tirages, appariement des itérations, exclusions des variantes non exécutables.

- **Contrat Python → R** (`tests/test_r_contract.py`) : les scripts R

lisent `results.json` via des clés implicites ; une clé renommée casserait les figures *silencieusement*. Ce fichier exécute un cas d'étude complet puis vérifie que **chaque chemin de clé** déréférencé par les scripts R existe avec la structure attendue (`uncertainty.emissions_table`, `source_gas_columns`, `central_gwp_factors`, `convention_sim_id`, appariement des tables..., et la grille multi-slots : `combinations`, `slots`, `main_selection`, `mode`). Si vous ajoutez une sortie consommée par R, ajoutez-y le test correspondant.

Étape 5 — Documentation auto-générée

```
pdoc pblca -o docs_html      # génère docs_html/ (HTML statique)
pdoc pblca                   # sert la doc localement (navigateur)
```

Ouvrir `docs_html/index.html` ou

`docs_html/pblca.html`. La documentation est

extraite des docstrings : chaque classe, méthode et fonction publique y a

son ancre. Liens utiles :

- [LCAEngine](#) — orchestration :

```
run,  
run_monte_carlo,  
run_ration_comparison,  
run_paired_variant_grid,  
run_paired_grid
```

- [load_card / run_card](#) —

card d'étude TOML (validation, exécution, traçabilité) et

[PairedGridConfig](#)

- [ParameterSet](#) — paramètres tracés :

```
draw,  
central_values
```

- [GasLedger](#) — inventaire traçable :

```
add,  
total,  
total_by_source
```

- [characterize](#) — impacts (couche 3) ;

```
GwpStarInputs,  
compute_gwpstar
```

- [SimulationResult](#) — résultat

d'une simulation (impacts, ledger, traces par slot)

- [DataStore](#) — stockage JSON et

archivage

- [ModelSpec](#) — fiche d'une variante

(slot, tier, référence, champs requis)

Règle du dépôt : `pytest` et `pdoc pblca -o docs_html` doivent passer avant chaque commit (AGENTS.md). Après toute modification de ce guide, régénérer le PDF : `python scripts/make_guide_pdf.py` (GUIDE.md reste la source de vérité). Les docstrings étant en anglais (convention du dépôt), la doc générée est en anglais ; ce guide, lui, est en français.

Pour aller plus loin (niveau développeur)

Mécanisme des tirages appariés (`pblca/mc.py`, `run_paired_grid` ;

`run_paired_variant_grid` en est le cas particulier entérique). À chaque itération :

1. `engine.params.draw(rng)` tire **toutes** les valeurs de paramètres

(y compris les facteurs GWP) ;

2. les rations mesurées sont perturbées (un facteur log-normal par groupe appliqué à DMI et GE, un par méthode de CH₄ mesuré) — *les mêmes mesures perturbées pour toutes les variantes*, donc l'appariement tient aussi pour les variantes *_ingestion_measured et measured_ahcs ;

3. chaque variante est évaluée avec ce tirage commun et ses indicateurs remplissent la ligne.

Conséquence : $\text{Var}(\text{colonne_A} - \text{colonne_B})$ ne contient que l'effet du choix de modèle — d'où la figure d'effet-modèle et les différences appariées de l'étape 3.1.

Décomposition exacte de variance (R/analyse_enteric_sensitivity.R, section 3). Le GWP100 d'une itération est exactement la somme des termes $t_{\{\text{source}, \text{gaz}\}} = \text{kg}_{\{\text{source}, \text{gaz}\}} \times \text{facteur_gwp}(\text{gaz})$. Chaque terme est scindé en deux : $\text{kg} \times \text{facteur_central}$ (erreur d'inventaire) et $\text{kg} \times (\text{facteur_tiré} - \text{facteur_central})$ (erreur de caractérisation ; nul pour le CO₂ dont le facteur vaut 1 par définition). Comme $\text{Var}(X) = \sum_i \text{Cov}(t_i, X)$ quand $X = \sum_i t_i$, les contributions $\text{Cov}(t_i, X)/\text{Var}(X)$ somment à 100 %. C'est une identité : pas de linéarisation, pas d'hypothèse d'indépendance, et les covariances négatives (termes qui se compensent) s'affichent telles quelles.

Paramètres de caractérisation (pblca/params.py) : les facteurs GWP sont rentrés comme les **valeurs tabulées de la table 7.15 AR6** (valeur centrale $\pm$ écart-type), convertis en paramètres log-normaux *dans le code* — on ne rentre jamais les paramètres μ/σ dérivés. Règle du dépôt : aucune valeur interpolée quand la source fournit une valeur tabulée.

Ajouter une variante : déclarer le modèle dans le slot concerné (pblca/processes/..., enregistré dans pblca/registry.py avec sa ModelSpec : référence, champs requis), puis l'ajouter à variant_grid dans case_studies/ferme_20ha.py. Elle sera automatiquement balayée, appariée et analysée ; si elle exige des champs de groupe absents de la ferme, elle est exclue avec la raison plutôt que d'échouer.

Ajouter une analyse R : vérifier les clés consommées dans tests/test_r_contract.py (et y ajouter les nouvelles), documenter les sorties dans R/README.md, et respecter la convention $\text{sim_id} = \text{mc}_{\langle \text{cas} \rangle}_{\langle \text{slot} \rangle}_{\langle \text{variante} \rangle}$ pour le regroupement des entrées.

Ajouter une ferme intégrée utilisable par card : décrire la ferme

en FarmSpec (valeurs seulement, cf. `pblca/case_study.py`),
l'enregistrer dans BUILTIN_FARMS (`pblca/card.py`), puis
`pblca farms` la liste et `farm = "..."` dans `[study]` la référence.
Les sections de la card étant validées au chargement, une faute de
frappe dans un slot ou une variante échoue immédiatement avec la liste
des valeurs acceptées.